

Currying and the Lambda Calculus

CS 350

Dr. Joseph Eremondi

May 7, 2025

Overview

Currying

Lambda The Ultimate

Overview

Objectives

- Learning Goals

Objectives

- Learning Goals
 - To learn how multi-argument functions can be desugared into single-argument functions

Objectives

- Learning Goals
 - To learn how multi-argument functions can be desugared into single-argument functions
 - Curly-Curry

- Learning Goals
 - To learn how multi-argument functions can be desugared into single-argument functions
 - Curly-Curry
 - To see that *everything* can be desugared into single-argument functions

- Learning Goals
 - To learn how multi-argument functions can be desugared into single-argument functions
 - Curly-Curry
 - To see that *everything* can be desugared into single-argument functions
 - by learning about the Lambda Calculus

Objectives

- Learning Goals
 - To learn how multi-argument functions can be desugared into single-argument functions
 - Curly-Curry
 - To see that *everything* can be desugared into single-argument functions
 - by learning about the Lambda Calculus
- Core Concepts

Objectives

- Learning Goals
 - To learn how multi-argument functions can be desugared into single-argument functions
 - Curly-Curry
 - To see that *everything* can be desugared into single-argument functions
 - by learning about the Lambda Calculus
- Core Concepts
 - Currying

Objectives

- Learning Goals
 - To learn how multi-argument functions can be desugared into single-argument functions
 - Curly-Curry
 - To see that *everything* can be desugared into single-argument functions
 - by learning about the Lambda Calculus
- Core Concepts
 - Currying
 - Lambda Calculus

Currying

Executing a multiple-argument function

- Say we allow $\{\text{lam } \{x \ y \ z\} \ \{+ \ x \ \{* \ y \ z\}\}\}$ in Curly

Executing a multiple-argument function

- Say we allow $\{\text{lam } \{x \ y \ z\} \ \{+ \ x \ \{* \ y \ z\}\}\}$ in Curly
- How can we interpret a call to this function?

Executing a multiple-argument function

- Say we allow $\{\text{lam } \{x \ y \ z\} \ \{+ \ x \ \{* \ y \ z\}\}\}$ in Curly
- How can we interpret a call to this function?
 - x, y, z replaced by concrete argument values (substitution)

Achieving this: Currying

- We can achieve this with nested lambda expressions

Achieving this: Currying

- We can achieve this with nested lambda expressions

Achieving this: Currying

- We can achieve this with nested lambda expressions

```
{let f {lam x {lam y {lam z {+ x {* y z}}}}}
....}
```

- To call, we do nested calls

Achieving this: Currying

- We can achieve this with nested lambda expressions

```
{let f {lam x {lam y {lam z {+ x {* y z}}}}}
....}
```

- To call, we do nested calls

Achieving this: Currying

- We can achieve this with nested lambda expressions

```
{let f {lam x {lam y {lam z {+ x {* y z}}}}}
....}
```

- To call, we do nested calls

```
{{{f 1} 2} 3}
```

- Step by step:

Achieving this: Currying

- We can achieve this with nested lambda expressions

```
{let f {lam x {lam y {lam z {+ x {* y z}}}}}
....}
```

- To call, we do nested calls

```
{{{f 1} 2} 3}
```

- Step by step:
 - $f\ 1$ produces $\{\text{lam } y \{\text{lam } z \{+ \ 1 \ \{* \ y \ z\}\}\}\}$

Achieving this: Currying

- We can achieve this with nested lambda expressions

```
{let f {lam x {lam y {lam z {+ x {* y z}}}}}
....}
```

- To call, we do nested calls

```
{{{f 1} 2} 3}
```

- Step by step:
 - $f\ 1$ produces $\{\text{lam } y\ \{\text{lam } z\ \{+ 1\ \{* y\ z\}\}\}$
 - Calling that on 2 produces $\{\text{lam } z\ \{+ 1\ \{* 2\ z\}\}$

Achieving this: Currying

- We can achieve this with nested lambda expressions

```
{let f {lam x {lam y {lam z {+ x {* y z}}}}}
....}
```

- To call, we do nested calls

```
{{{f 1} 2} 3}
```

- Step by step:
 - $f\ 1$ produces $\{\text{lam } y\ \{\text{lam } z\ \{+ 1\ \{* y\ z\}\}\}$
 - Calling that on 2 produces $\{\text{lam } z\ \{+ 1\ \{* 2\ z\}\}$
 - Calling that on 3 produces $\{+ 1\ \{* 2\ 3\}\}$

Achieving this: Currying

- We can achieve this with nested lambda expressions

```
{let f {lam x {lam y {lam z {+ x {* y z}}}}}
....}
```

- To call, we do nested calls

```
{{{f 1} 2} 3}
```

- Step by step:
 - `f 1` produces `{lam y {lam z {+ 1 {* y z}}}}`
 - Calling that on 2 produces `{lam z {+ 1 {* 2 z}}}`
 - Calling that on 3 produces `{+ 1 {* 2 3}}`
 - Exactly what we want for `{f 1 2 3}`



- The approach of simulating multiple-argument functions with nested single-argument functions is called **Currying**



- The approach of simulating multiple-argument functions with nested single-argument functions is called **Currying**
 - Named after Haskell Curry, and American logician



- The approach of simulating multiple-argument functions with nested single-argument functions is called **Currying**
 - Named after Haskell Curry, and American logician
 - Also the namesake of the Haskell programming language



- The approach of simulating multiple-argument functions with nested single-argument functions is called **Currying**
 - Named after Haskell Curry, and American logician
 - Also the namesake of the Haskell programming language
- A function written in this style is *curried*

- We can add multiple-argument functions to our Surface Language *without* changing the core language

Desugaring with Currying

- We can add multiple-argument functions to our Surface Language *without* changing the core language
 - Handle in desugar

AST for Multi-Argument Functions

AST for Multi-Argument Functions

```
(define-type ExpS
  ....
  (lamS [xs : (Listof Symbol)]
        [body : ExpS]))
(appS [fun : ExpS]
      [args : (Listof ExpS)])
```

- Functions now take a list of variables

AST for Multi-Argument Functions

```
(define-type ExpS
  ....
  (lamS [xs : (Listof Symbol)]
        [body : ExpS]))
  (appS [fun : ExpS]
        [args : (Listof ExpS)])
```

- Functions now take a list of variables
- Calls now take a list of arguments

Elaborating Multi-Argument Functions

- `lamE` has type `(Symbol Exp -> Exp)`

Elaborating Multi-Argument Functions

- `lamE` has type `(Symbol Exp -> Exp)`
 - Can use as argument to `fold`!

Elaborating Multi-Argument Functions

- `lamE` has type `(Symbol Exp -> Exp)`
 - Can use as argument to `fold`!

Elaborating Multi-Argument Functions

- `lamE` has type `(Symbol Exp -> Exp)`
 - Can use as argument to fold!

```
(define (desugar [se : ExpS])  
  (type-case ExpS se  
    ....  
    [(lamS xs body)  
     (foldr lamE (desugar body) xs)  
     ]))
```

- No arguments: just produce the body

Elaborating Multi-Argument Functions

- `lamE` has type `(Symbol Exp -> Exp)`
 - Can use as argument to fold!

```
(define (desugar [se : ExpS])  
  (type-case ExpS se  
    ....  
    [(lamS xs body)  
     (foldr lamE (desugar body) xs)  
     ]))
```

- No arguments: just produce the body
- At least one argument: curry the rest of the arguments, and wrap the result in a lambda

Elaborating Multi-Argument Functions

- `lamE` has type `(Symbol Exp -> Exp)`
 - Can use as argument to fold!

```
(define (desugar [se : ExpS])  
  (type-case ExpS se  
    ....  
    [(lamS xs body)  
     (foldr lamE (desugar body) xs)  
     ]))
```

- No arguments: just produce the body
- At least one argument: curry the rest of the arguments, and wrap the result in a lambda
 - `(lamS '(x y z) body)` becomes `(lamE 'x (lamE 'y (lamE 'z body)))`

Multi-Argument Calls

- Function calls associate in the opposite direction

Multi-Argument Calls

- Function calls associate in the opposite direction
 - So need to fold left

Multi-Argument Calls

- Function calls associate in the opposite direction
 - So need to fold left

Multi-Argument Calls

- Function calls associate in the opposite direction
 - So need to fold left

```
(define (desugar [surfExpr : ExpS])  
  (type-case ExpS surfExpr  
    ....  
    [(appS funExpr args)  
     (foldl appE (desugar funExpr) args)]))
```

- Given a list of argument to apply, build up one giant expression with nested calls

Multi-Argument Calls

- Function calls associate in the opposite direction
 - So need to fold left

```
(define (desugar [surfExpr : ExpS])  
  (type-case ExpS surfExpr  
    ....  
    [(appS funExpr args)  
     (foldl appE (desugar funExpr) args)]))
```

- Given a list of argument to apply, build up one giant expression with nested calls
- {f a b c} becomes {{{f a} b} c}

Putting Them Together

- To run `{{lam {x y z} body} a b c}`:

Putting Them Together

- To run `{{lam {x y z} body} a b c}`:
 - Desugared into `{{{{{lam x {lam y {lam z body}}}} a} b} c}`

Putting Them Together

- To run `{{lam {x y z} body} a b c}`:
 - Desugared into `{{{{{lam x {lam y {lam z body}}}} a} b} c}`
 - Interp makes `(subst x a {lam y {lam z body}})`

Putting Them Together

- To run `{{lam {x y z} body} a b c}`:
 - Desugared into `{{{{lam x {lam y {lam z body}}}} a} b} c}`
 - Interp makes `(subst x a {lam y {lam z body}})`
 - then `(subst x a (subst y b {lam z body}))`

Putting Them Together

- To run `{{lam {x y z} body} a b c}`:
 - Desugared into `{{{{lam x {lam y {lam z body}}}} a} b} c}`
 - Interp makes `(subst x a {lam y {lam z body}})`
 - then `(subst x a (subst y b {lam z body}))`
 - then `(subst x a (subst y b (subst z c body)))`

A Word of Warning

- Flit/Racket do *not* use currying by default

A Word of Warning

- Flit/Racket do *not* use currying by default
 - Multiple argument functions are *not* desugared into single-argument ones

A Word of Warning

- Flit/Racket do *not* use currying by default
 - Multiple argument functions are *not* desugared into single-argument ones
- We can convert between curried and uncurried functions with combinators

A Word of Warning

- Flit/Racket do *not* use currying by default
 - Multiple argument functions are *not* desugared into single-argument ones
- We can convert between curried and uncurried functions with combinators
 - See lecture on Higher Order Functions

A Word of Warning

- Flit/Racket do *not* use currying by default
 - Multiple argument functions are *not* desugared into single-argument ones
- We can convert between curried and uncurried functions with combinators
 - See lecture on Higher Order Functions
- A 0-argument `(lambda () e)` is *NOT* the same as `e` in Flit

A Word of Warning

- Flit/Racket do *not* use currying by default
 - Multiple argument functions are *not* desugared into single-argument ones
- We can convert between curried and uncurried functions with combinators
 - See lecture on Higher Order Functions
- A 0-argument `(lambda () e)` is *NOT* the same as `e` in Flit
 - But it is in Curly, if we use this desugaring

Another Desugaring: Let

- Recall that `let` let us define a local variable to have the value of some expression, that we could then use to build another expression

Another Desugaring: Let

- Recall that `let` let us define a local variable to have the value of some expression, that we could then use to build another expression
- We can make `let` syntactic sugar using `lambda`

Another Desugaring: Let

- Recall that `let` let us define a local variable to have the value of some expression, that we could then use to build another expression
- We can make `let` syntactic sugar using `lambda`

Another Desugaring: Let

- Recall that `let` let us define a local variable to have the value of some expression, that we could then use to build another expression
- We can make `let` syntactic sugar using lambda

```
(define (desugar [surfExpr : ExpS])  
  (type-case ExpS surfExpr  
    ....  
    [(let1S x xExpr body)  
     (appE (lamE x (desugar body))  
            (desugar xExpr))]))
```

- Define a function whose body is the body of the `let`

Another Desugaring: Let

- Recall that `let1` let us define a local variable to have the value of some expression, that we could then use to build another expression
- We can make `let1` syntactic sugar using lambda

```
(define (desugar [surfExpr : ExpS])  
  (type-case ExpS surfExpr  
    ....  
    [(let1S x xExpr body)  
     (appE (lamE x (desugar body))  
            (desugar xExpr))]))
```

- Define a function whose body is the body of the `let1`
- Immediately call it with the value we're giving to the defined variable

Another Desugaring: Let

- Recall that `let1` let us define a local variable to have the value of some expression, that we could then use to build another expression
- We can make `let1` syntactic sugar using lambda

```
(define (desugar [surfExpr : ExpS])  
  (type-case ExpS surfExpr  
    ....  
    [(let1S x xExpr body)  
     (appE (lamE x (desugar body))  
           (desugar xExpr))]))
```

- Define a function whose body is the body of the `let1`
- Immediately call it with the value we're giving to the defined variable
- Does the exact same thing as `Let1`

Lambda The Ultimate

Functions As Sugar

- So far we've seen that single argument functions can simulate

- So far we've seen that single argument functions can simulate
 - Multi-argument functions

- So far we've seen that single argument functions can simulate
 - Multi-argument functions
 - Local variable definitions

Functions As Sugar

- So far we've seen that single argument functions can simulate
 - Multi-argument functions
 - Local variable definitions
- What else can se simulate?

Functions As Sugar

- So far we've seen that single argument functions can simulate
 - Multi-argument functions
 - Local variable definitions
- What else can se simulate?
- **NOTE** I won't ask about the following desugarings on an exam

- So far we've seen that single argument functions can simulate
 - Multi-argument functions
 - Local variable definitions
- What else can se simulate?
- **NOTE** I won't ask about the following desugarings on an exam
 - But they're an important introduction to the “science” of computer science and the “mathematics” of informatics

The Smallest Language We Can Imagine

The Smallest Language We Can Imagine

```
(define-type UTLC
  ;; A variable
  (varE [x : Symbol])
  ;; Function application (call)
  (appE [fun : UTLC]
        [arg : UTLC])
  ;; Anonymous function (lambda)
  (lamE [param : Symbol]
        [body : UTCL]))
```

- Stands for “Un-Typed Lambda Calculus”

The Smallest Language We Can Imagine

```
(define-type UTLC
  ;; A variable
  (varE [x : Symbol])
  ;; Function application (call)
  (appE [fun : UTLC]
        [arg : UTLC])
  ;; Anonymous function (lambda)
  (lamE [param : Symbol]
        [body : UTCL]))
```

- Stands for “Un-Typed Lambda Calculus”
- All you can do is define an anonymous function (lambda) or call a function (application)

The Smallest Language We Can Imagine

```
(define-type UTLC
  ;; A variable
  (varE [x : Symbol])
  ;; Function application (call)
  (appE [fun : UTLC]
        [arg : UTLC])
  ;; Anonymous function (lambda)
  (lamE [param : Symbol]
        [body : UTCL]))
```

- Stands for “Un-Typed Lambda Calculus”
- All you can do is define an anonymous function (lambda) or call a function (application)
- **The Untyped Lambda Calculus is Turing Complete**

The Smallest Language We Can Imagine

```
(define-type UTLC
  ;; A variable
  (varE [x : Symbol])
  ;; Function application (call)
  (appE [fun : UTLC]
        [arg : UTLC])
  ;; Anonymous function (lambda)
  (lamE [param : Symbol]
        [body : UTCL]))
```

- Stands for “Un-Typed Lambda Calculus”
- All you can do is define an anonymous function (lambda) or call a function (application)
- **The Untyped Lambda Calculus is Turing Complete**
 - Any program you can write, you can write an equivalent UTLC Program

Interpreting the UTLC

- Works just like we've seen so far

Interpreting the UTLC

- Works just like we've seen so far
 - Lam is like Fun

Interpreting the UTLC

- Works just like we've seen so far
 - Lam is like Fun
 - App is like Call

Interpreting the UTLC

- Works just like we've seen so far
 - Lam is like Fun
 - App is like Call
- Substitution version

Interpreting the UTLC

- Works just like we've seen so far
 - Lam is like Fun
 - App is like Call
- Substitution version

Interpreting the UTLC

- Works just like we've seen so far
 - Lam is like Fun
 - App is like Call
- Substitution version

```
(define (interpUTLC [e : UTLC] : UTLC)
  (type-case UTLC e
    [(appE fun arg)
     (type-case (interpUTLC fun)
       [(lamE x body)
        (interpUTLC (subst x arg body))]
       [else (error 'x "Undefined variable")])]
    ;; Function and variables don't do any computation,
    ;; they just return themselves
    ;; Could have variables return an error,
    [else e]))
```

- Can also do with environments, like Curly-Lambda-Env

- Note: I'll write the desugarings as functions, rather than with `elab`, just to keep things self contained

- Note: I'll write the desugarings as functions, rather than with `elab`, just to keep things self contained
- You can imagine these functions as constructors we'd use for the surface syntax, that produce UTLC directly

- Note: I'll write the desugarings as functions, rather than with `elab`, just to keep things self contained
- You can imagine these functions as constructors we'd use for the surface syntax, that produce UTLC directly

Booleans

- Note: I'll write the desugarings as functions, rather than with `eLab`, just to keep things self contained
- You can imagine these functions as constructors we'd use for the surface syntax, that produce UTLC directly

```
;; {lam {x y} x}
(define True
  (lamE 'x (lamE 'y (varE 'x))))
;; {lam {x y} y}
(define False
  (lamE 'x (lamE 'y (varE 'y))))
;; curried (test thenCase elseCase)
(define (If test thenCase elseCase)
  (appE (appE test thenCase) elseCase))
```

- If we give `If` the boolean `True` it produces the then-case

Booleans

- Note: I'll write the desugarings as functions, rather than with `eLab`, just to keep things self contained
- You can imagine these functions as constructors we'd use for the surface syntax, that produce UTLC directly

```
;; {lam {x y} x}
(define True
  (lamE 'x (lamE 'y (varE 'x))))
;; {lam {x y} y}
(define False
  (lamE 'x (lamE 'y (varE 'y))))
;; curried (test thenCase elseCase)
(define (If test thenCase elseCase)
  (appE (appE test thenCase) elseCase))
```

- If we give `If` the boolean `True` it produces the then-case
- If we give `If` the boolean `False` it produces the else-case

- Define n to be the function that takes in another function and an argument, and applies it n times

- Define n to be the function that takes in another function and an argument, and applies it n times
 - $\lambda f \lambda x. f(f(f \dots (fx)))$

- Define n to be the function that takes in another function and an argument, and applies it n times
 - $\lambda f \lambda x. f(f(f \dots (fx)))$

Numbers

- Define n to be the function that takes in another function and an argument, and applies it n times

- $\lambda f \lambda x. f(f(f \dots (fx)))$

```
;; {lam {f x} x}
(define Zero
  (lamE 'f (lamE 'x (varE 'x)))) ;; Function that returns its argument
;; {lam {f x} {f {n f x}}}
;; Add one to a number
(define (Add1 n)
  (lamE 'f (lamE 'x) (appE (varE 'f) (appE (appE n f) x))))
;; {lam {f x} {m f {n f x}}}
(define (plusE m n)
  (lamE 'f (lamE 'x) ((appE m (varE 'f)) (appE (appE n f) x))))
```


- You can view a pair as a function that takes in a boolean and returns either the first or second value depending on that boolean

- You can view a pair as a function that takes in a boolean and returns either the first or second value depending on that boolean

- You can view a pair as a function that takes in a boolean and returns either the first or second value depending on that boolean

```
;; {lam z {if z x y}} for the pair (x,y)  
(define (Pair x y)  
  (lamE 'z (If (varE 'z) x y)))  
;; {pr true} gives the first element of pr  
(define (Fst pr)  
  (appE pr True))  
;; {pr false} gives the second element of pr  
(define (Snd pr)  
  (appE pr False))
```

Recursion: The Y Combinator

- Not the startup funder

Recursion: The Y Combinator

- Not the startup funder
- $\lambda f. (\lambda x. f (x x))(\lambda x. f (x x))$

Recursion: The Y Combinator

- Not the startup funder
- $\lambda f. (\lambda x. f (x x))(\lambda x. f (x x))$

Recursion: The Y Combinator

- Not the startup funder
- $\lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))$

```
;;{lam f {{lam x {f {x x}}}} {lam x {f {x x}}}}}  
(define Y  
  (lamE 'f (appE (lamE 'x (appE f (appE x x)))  
                 (lamE 'x (appE f (appE x x))))))
```

- Fixed-point function

Recursion: The Y Combinator

- Not the startup funder
- $\lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))$

```
;;{lam f {{lam x {f {x x}}}} {lam x {f {x x}}}}}  
(define Y  
  (lamE 'f (appE (lamE 'x (appE f (appE x x)))  
                  (lamE 'x (appE f (appE x x))))))
```

- Fixed-point function
 - For any g, have $\{Y \ g\} = \{g \ \{Y \ g\}\} = \{g \ \{g \ \{Y \ g\}\}\} \dots$

Recursion: The Y Combinator

- Not the startup funder
- $\lambda f. (\lambda x. f (x x))(\lambda x. f (x x))$

```
;;{lam f {{lam x {f {x x}}}} {lam x {f {x x}}}}}  
(define Y  
  (lamE 'f (appE (lamE 'x (appE f (appE x x)))  
                  (lamE 'x (appE f (appE x x))))))
```

- Fixed-point function
 - For any g, have $\{Y\ g\} = \{g\ \{Y\ g\}\} = \{g\ \{g\ \{Y\ g\}\}\} \dots$
 - e.g. replace g's first argument with g, infinitely

Recursion: The Y Combinator

- Not the startup funder
- $\lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))$

```
;;{lam f {{lam x {f {x x}}}} {lam x {f {x x}}}}}  
(define Y  
  (lamE 'f (appE (lamE 'x (appE f (appE x x)))  
                 (lamE 'x (appE f (appE x x))))))
```

- Fixed-point function
 - For any g , have $\{Y\ g\} = \{g\ \{Y\ g\}\} = \{g\ \{g\ \{Y\ g\}\}\} \dots$
 - e.g. replace g 's first argument with g , infinitely
- Takes a function f with an extra parameter `self`

Recursion: The Y Combinator

- Not the startup funder
- $\lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))$

```
;;{lam f {{lam x {f {x x}}}} {lam x {f {x x}}}}}  
(define Y  
  (lamE 'f (appE (lamE 'x (appE f (appE x x)))  
                  (lamE 'x (appE f (appE x x))))))
```

- Fixed-point function
 - For any g , have $\{Y \ g\} = \{g \ \{Y \ g\}\} = \{g \ \{g \ \{Y \ g\}\}\} \dots$
 - e.g. replace g 's first argument with g , infinitely
- Takes a function f with an extra parameter `self`
- Makes a new function where each call to `self` is replaced by a call to f

Recursion: The Y Combinator

- Not the startup funder
- $\lambda f. (\lambda x. f (x x))(\lambda x. f (x x))$

```
;;{lam f {{lam x {f {x x}}}} {lam x {f {x x}}}}}  
(define Y  
  (lamE 'f (appE (lamE 'x (appE f (appE x x)))  
                 (lamE 'x (appE f (appE x x))))))
```

- Fixed-point function
 - For any g , have $\{Y \ g\} = \{g \ \{Y \ g\}\} = \{g \ \{g \ \{Y \ g\}\}\} \dots$
 - e.g. replace g 's first argument with g , infinitely
- Takes a function f with an extra parameter `self`
- Makes a new function where each call to `self` is replaced by a call to f
- You don't need to know the details of how this work, just that it's possible to do recursion in the UTLC

Recursion: The Y Combinator

- Not the startup funder
- $\lambda f. (\lambda x. f (x x))(\lambda x. f (x x))$

```
;;{lam f {{lam x {f {x x}}}} {lam x {f {x x}}}}}  
(define Y  
  (lamE 'f (appE (lamE 'x (appE f (appE x x)))  
                 (lamE 'x (appE f (appE x x))))))
```

- Fixed-point function
 - For any g , have $\{Y\ g\} = \{g\ \{Y\ g\}\} = \{g\ \{g\ \{Y\ g\}\}\} \dots$
 - e.g. replace g 's first argument with g , infinitely
- Takes a function f with an extra parameter `self`
- Makes a new function where each call to `self` is replaced by a call to f
- You don't need to know the details of how this work, just that it's possible to do recursion in the UTLC
- Once we have recursion, we have loops